

المدرسة الوطنية للعلوم التطبيقية
École Nationale des Sciences Appliquées d'Oujda

École Nationale des Sciences
Appliquées Oujda

Filière Génie Informatique Option IA



المدرسة الوطنية للعلوم التطبيقية
École Nationale des Sciences Appliquées d'Oujda

RAPPORT DE PROJET DE FIN D'ANNEE

SmarTest

*Système Intelligent Sécurisé de Gestion des Quiz et
Examens*

Réalisé par :

CHAHMAL Ikram
EL AAMMARI Oumeyma
EL MNIAI Nissrine
HAMDAOUI Lamyae

Sous la direction de :

M. BOUCHENTOUF Toumi
M. AGHBAL Othmane

Année universitaire : 2025 – 2026

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien constant et leurs sacrifices, qui ont permis la poursuite de nos études dans les meilleures conditions.

À nos encadrants, M. BOUCHENTOUF Toumi et M. AGHBAL Othmane, pour la qualité de leur accompagnement, la rigueur de leurs conseils et leur disponibilité tout au long de ce projet.

À l'ensemble des enseignants et du personnel de l'École Nationale des Sciences Appliquées d'Oujda, pour leur contribution à notre formation académique et professionnelle.

À nos collègues et camarades de promotion, pour les échanges fructueux et l'entraide qui ont enrichi notre travail.

À toutes les personnes qui, par leur soutien ou leur expertise, ont contribué à la réalisation de ce projet.

Ce rapport témoigne de leur apport et de notre engagement collectif.

Remerciements

La réalisation de ce projet n'aurait pu aboutir sans le soutien et l'accompagnement de nombreuses personnes auxquelles nous souhaitons exprimer notre profonde gratitude.

Nous remercions avant tout Allah, le Tout-Puissant, pour nous avoir accordé la santé, la volonté et la patience nécessaires afin de mener ce travail à bien.

Nous tenons à adresser nos sincères remerciements à nos encadrants, M. BOUCHENTOUF Toumi et M. AGHBAL Othmane, pour leur disponibilité, leurs précieux conseils, leur suivi rigoureux ainsi que la confiance qu'ils nous ont accordée tout au long de ce projet.

Nos remerciements vont également à l'ensemble du corps professoral et administratif de l'École Nationale des Sciences Appliquées d'Oujda, pour la qualité de la formation dispensée et l'environnement favorable à notre apprentissage.

Nous exprimons aussi notre reconnaissance envers nos familles et nos proches pour leur soutien moral, leurs encouragements permanents et leur présence durant tout notre parcours universitaire.

Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce travail et au bon déroulement de cette expérience enrichissante.

Qu'ils trouvent ici l'expression de notre profonde reconnaissance et de notre respect.

Résumé

Le développement des technologies numériques dans le secteur éducatif a transformé les pratiques d'évaluation, mais les solutions existantes présentent des limites en matière de sécurité des données pédagogiques, de personnalisation et de contrôle en temps réel. Face à ce constat, ce projet propose **SmarTest**, une plateforme intelligente et sécurisée de gestion des quiz et examens.

SmarTest repose sur une architecture hybride combinant une application desktop pour les professeurs, une interface web pour les étudiants, et un backend centralisé assurant la communication entre les deux interfaces en temps réel. L'originalité du système réside dans l'intégration d'un service d'inférence basé sur les grands modèles de langage (LLM), permettant la génération automatique de questions (QCM, vrai/faux, rédaction) à partir des cours importés par l'enseignant, sans que les données pédagogiques ne quittent sa machine.

Les fonctionnalités principales incluent : l'import de documents pédagogiques, la génération paramétrable de questions, la création et la supervision d'examens en temps réel, la correction automatique, ainsi que la consultation des résultats avec des statistiques détaillées. La sécurité constitue un axe majeur du projet : les examens sont stockés localement sur le poste du professeur, seules les métadonnées fonctionnelles transitent vers le backend, et l'accès est sécurisé par authentification.

Ce rapport présente l'ensemble du cycle de développement, de l'étude de l'existant et l'analyse des besoins à la conception UML, en passant par la réalisation et les tests de validation. Les résultats démontrent la faisabilité d'une plateforme alliant intelligence artificielle générative, sécurité des données et contrôle pédagogique renforcé.

Mots-clés : SmarTest, Quiz, Examens, Génération automatique de questions, Intelligence Artificielle, Sécurité des données, Supervision en temps réel, ENSA Oujda

Abstract

The development of digital technologies in the education sector has transformed assessment practices, but existing solutions have limitations regarding pedagogical data security, customization, and real-time control. In response, this project presents **SmarTest**, an intelligent and secure quiz and exam management platform.

SmarTest is built on a hybrid architecture combining a desktop application for professors, a web interface for students, and a centralized backend ensuring real-time communication between both interfaces. The system's originality lies in the integration of a fast inference service based on large language models (LLMs), enabling automatic question generation (MCQ, true/false, short answer) from course materials imported by the teacher, without any pedagogical data leaving their machine.

The main features include : document import, parameterizable question generation, real-time exam creation and supervision, automatic correction, and detailed result statistics. Security is a major focus : exams are stored locally on the professor's machine, only functional metadata is transmitted to the backend, and access is secured by authentication.

This report presents the entire development cycle, from requirements analysis to UML design, implementation, and validation testing. The results demonstrate the feasibility of a platform combining generative artificial intelligence, data security, and enhanced pedagogical control.

Keywords : SmarTest, Quiz, Exams, Automatic question generation, Artificial Intelligence, Data security, Real-time supervision, ENSA Oujda

Table des figures

3.1	Architecture technique globale de SmarTest	9
3.2	Diagramme des cas d'utilisation — SmarTest	10
3.3	Diagramme de séquence global du système SmarTest	11
3.4	Diagramme de classes global du système SmarTest	12
4.1	Flux des statistiques pour un quiz persisté	21

Liste des tableaux

2.1	Comparaison des solutions existantes	5
2.2	Besoins non fonctionnels	6
2.3	Résumé des rôles du professeur	7
2.4	Résumé des rôles de l'étudiant	7
2.5	Interactions acteurs – composantes du système	7
3.1	Cas d'utilisation par acteur	10
3.2	Modules de l'application desktop (Professeur)	12
3.3	Écrans de l'application web (Étudiant)	13
4.1	Stack technologique globale du projet	14
4.2	Choix technologiques et justifications	15
4.3	Séparation des données lors de la publication	16
4.4	Types de questions et méthode de correction	18
4.5	Récapitulatif de l'implémentation par composante	18
4.6	Seuils d'alerte — comparaison backend / interface desktop	20
5.1	Récapitulatif des résultats de tests	23
6.1	Bilan des objectifs non fonctionnels	25
6.2	Synthèse des fonctionnalités réalisées	26

Liste des abréviations

API	Application Programming Interface
ENSA	École Nationale des Sciences Appliquées
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
MVVM	Model-View-ViewModel
QCM	Question à Choix Multiple
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
WPF	Windows Presentation Foundation

Table des matières

Dédicaces	ii
Remerciements	iii
Résumé	iv
Abstract	v
Liste des figures	vi
Liste des tableaux	vii
Liste des abréviations.....	viii
1 Introduction Générale	1
1.1 Contexte et motivation	1
1.2 Problématique	1
1.3 Objectifs du projet.....	2
1.3.1 <i>Objectif principal</i>	2
1.3.2 <i>Côté professeur — application desktop</i>	2
1.3.3 <i>Côté étudiant — application web</i>	2
1.3.4 <i>Objectifs non fonctionnels transversaux</i>	3
1.4 Périmètre et limites du projet.....	3
1.4.1 <i>Ce que le projet couvre</i>	3
1.4.2 <i>Ce que le projet ne couvre pas</i>	3
1.5 Structure du rapport	3
2 Étude de l’Existant et Analyse des Besoins.....	4
2.1 Étude de l’existant.....	4
2.1.1 <i>Solutions existantes</i>	4
2.1.2 <i>Tableau comparatif critique</i>	5
2.1.3 <i>Limites identifiées</i>	5
2.2 Analyse des besoins	5
2.2.1 <i>Besoins fonctionnels</i>	5
2.2.2 <i>Besoins non fonctionnels</i>	6
2.3 Identification des acteurs.....	6
2.3.1 <i>Professeur</i>	7
2.3.2 <i>Étudiant</i>	7
2.3.3 <i>Synthèse des interactions</i>	7
3 Conception du Système.....	8
3.1 Architecture globale du système	8
3.1.1 <i>Description des composants</i>	8
3.1.2 <i>Schéma d’architecture technique</i>	9
3.2 Modélisation UML.....	9
3.2.1 <i>Diagramme des cas d’utilisation</i>	9
3.2.2 <i>Diagramme de séquence</i>	10
3.2.3 <i>Diagramme de classes</i>	11
3.3 Conception des interfaces	12
3.3.1 <i>Côté professeur — application desktop</i>	12
3.3.2 <i>Côté étudiant — application web</i>	13

4	Réalisation et Implémentation	14
4.1	Technologies utilisées	14
4.2	Justification des choix technologiques	14
4.3	Fonctionnalités réalisées — Application Desktop	15
4.3.1	Import de documents pédagogiques	15
4.3.2	Génération automatique de questions via l'IA	16
4.3.3	Import de la liste des étudiants autorisés	16
4.3.4	Publication vers le backend	16
4.3.5	Supervision des sessions d'examen	16
4.3.6	Correction des examens	16
4.4	Fonctionnalités réalisées — Application Web	17
4.4.1	Authentification et tableau de bord	17
4.4.2	Passage des évaluations	17
4.4.3	Consultation des résultats	17
4.5	Fonctionnalités réalisées — Backend	17
4.5.1	Authentification et contrôle d'accès	17
4.5.2	Supervision des examens	17
4.5.3	Correction automatique	17
4.6	Gestion de la sécurité	18
4.6.1	Synthèse de l'implémentation	18
4.7	Statistiques et alertes pédagogiques	19
4.7.1	Statistiques de quiz (flux persisté)	19
4.7.2	Statistiques d'examen publié	19
4.7.3	Statistiques live QR (en mémoire)	20
4.7.4	Affichage desktop et seuils d'alerte	20
4.7.5	Synthèse du flux statistique (quiz persisté)	21
5	Tests et Validation	22
5.1	Stratégie de tests	22
5.2	Tests unitaires	22
5.3	Tests d'intégration	22
5.4	Tests fonctionnels	23
5.5	Tests de performance et de charge	23
5.6	Résultats et corrections apportées	23
6	Conclusion et Perspectives	24
6.1	Bilan du projet : objectifs atteints	24
6.1.1	Objectifs fonctionnels atteints	24
6.1.2	Objectifs non fonctionnels atteints	25
6.1.3	Synthèse	25
6.2	Difficultés rencontrées et solutions adoptées	26
6.2.1	Difficultés techniques	26
6.2.2	Difficultés organisationnelles	27
6.3	Perspectives d'évolution	28
6.3.1	Intégration d'un agent IA véritablement local	28
6.3.2	Application mobile	28
6.3.3	Mécanismes anti-triche avancés	28
6.3.4	Tableau de bord analytique enrichi	28
6.3.5	Gestion multi-professeurs	29
6.3.6	Amélioration de la génération de questions	29

Chapitre 1 | Introduction Générale

1.1 Contexte et motivation

Avec l'évolution rapide du numérique dans le domaine éducatif, les enseignants font face à des défis croissants dans la création, la gestion et l'analyse des évaluations pédagogiques. La massification des effectifs et la généralisation des formations hybrides ont rendu les méthodes traditionnelles — examens papier, corrections manuelles, tableurs de notes — à la fois coûteuses en temps et difficilement scalables.

L'intelligence artificielle générative, et plus particulièrement les grands modèles de langage (LLM), ouvre aujourd'hui de nouvelles perspectives pour automatiser la génération de questions pertinentes à partir de contenus pédagogiques, réduire la charge de travail des enseignants et améliorer la qualité des évaluations.

Cependant, la plupart des solutions existantes — Moodle, Kahoot!, Google Forms, Quizlet — reposent sur des plateformes génériques et centralisées qui ne répondent pas aux exigences spécifiques d'un établissement comme l'ENSA. Ces outils ne proposent pas de génération automatique de questions intégrée, offrent peu de contrôle sur le déroulement des examens en temps réel, et ne garantissent pas la confidentialité des données pédagogiques.

C'est dans ce contexte que s'inscrit **SmarTest**, une plateforme intelligente et sécurisée de gestion des quiz et examens, développée dans le cadre de la filière Génie Informatique option Ingénierie Logiciel et Intelligence Artificielle à l'ENSA. Le système repose sur une architecture hybride combinant une application desktop pour le professeur, une interface web pour l'étudiant, et un agent IA pour la génération automatique de questions. Les données pédagogiques sensibles — cours, questions, examens — restent stockées sur le poste du professeur, tandis que seules les métadonnées fonctionnelles transitent vers le backend central.

1.2 Problématique

La gestion des évaluations pédagogiques dans l'enseignement supérieur repose encore largement sur des pratiques manuelles ou des outils génériques inadaptés aux exigences actuelles. Cette réalité engendre une série de problèmes concrets auxquels les enseignants et les établissements sont confrontés au quotidien.

Sur le plan de la création des évaluations, la rédaction manuelle de questions reste chronophage. Les outils LLM existants sont soit trop génériques, soit dépourvus de garanties contractuelles sur la confidentialité des contenus pédagogiques transmis.

Sur le plan de l'organisation des examens, les solutions existantes ne permettent pas au professeur de garder un contrôle direct et en temps réel sur une session. Sur le plan de la correction, la correction manuelle reste chronophage et sujette à l'erreur humaine. Sur le plan de la confidentialité, aucune solution du marché ne propose une intégration garantissant contractuellement que les contenus pédagogiques transmis ne seront pas réutilisés à des fins d'entraînement de modèles.

Ces constats soulèvent une question centrale :

Comment exploiter la puissance des modèles de langage modernes pour générer des évaluations de qualité, tout en bénéficiant de garanties contractuelles sur la confidentialité des données pédagogiques et en assurant un contrôle total du professeur sur le déroulement des examens ?

C'est précisément à cette problématique que répond le projet **SmarTest**, en intégrant l'API Groq — dont le contrat de services interdit explicitement l'utilisation des données pour l'entraînement des modèles — et en offrant au professeur une supervision complète et en temps réel de ses sessions d'examen.

1.3 Objectifs du projet

Le projet **SmarTest** a pour ambition de concevoir et développer une plateforme complète, intelligente et sécurisée de gestion des quiz et examens.

1.3.1 Objectif principal

Développer une application hybride permettant la création, la gestion et le passage de quiz et examens de manière fluide, sécurisée et assistée par intelligence artificielle, tout en garantissant que les données pédagogiques sensibles restent sous le contrôle de l'enseignant.

1.3.2 Côté professeur — application desktop

L'application desktop vise à offrir à l'enseignant un environnement de travail autonome et complet. Elle lui permet de :

- **Importer des cours** au format PDF, DOCX ou TXT comme base de génération temporaire.
- **Générer des quiz** en définissant le nombre de questions, le niveau de difficulté et le type. Les questions sont générées automatiquement via l'IA à partir du contenu du cours, stockées exclusivement en local, et peuvent être modifiées manuellement avant publication.
- **Générer des examens** avec différents types de questions (QCM, vrai/faux, rédaction), en définissant la durée et en important obligatoirement une liste d'étudiants autorisés, garantissant un contrôle strict des accès.
- **Lancer et superviser les sessions d'examen en temps réel**, avec un contrôle direct sur le démarrage, le déroulement et la clôture de chaque épreuve. Le professeur visualise en temps réel la progression des étudiants.
- **Corriger les réponses de rédaction assisté par l'IA**, en gardant la main sur la validation ou la modification des notes suggérées avant enregistrement définitif.
- **Consulter et analyser les résultats**, avec des statistiques détaillées par question et par étudiant, et une alerte automatique lorsque le taux d'échec dépasse un seuil sur une question donnée.
- **Exporter les résultats** pour archivage ou transmission administrative.

1.3.3 Côté étudiant — application web

L'interface web destinée aux étudiants se veut simple, épurée et accessible sans installation préalable. Elle leur permet de :

- **S’authentifier de manière sécurisée** et accéder uniquement aux évaluations qui leur sont destinées.
- **Consulter un tableau de bord personnalisé** avec la liste des quiz et examens disponibles ainsi que le calendrier des épreuves planifiées.
- **Passer les quiz et examens** avec un affichage séquentiel question par question, un chronomètre dynamique et une soumission sécurisée des réponses.
- **Consulter leurs résultats et scores détaillés** dès la validation par le professeur.

1.3.4 Objectifs non fonctionnels transversaux

Au-delà des fonctionnalités, SmarTest vise à satisfaire des exigences de qualité essentielles : sécurité et confidentialité des données, performance et réactivité, ergonomie et expérience utilisateur, ainsi que fiabilité et résilience du système durant les sessions d’examen.

1.4 Périmètre et limites du projet

1.4.1 Ce que le projet couvre

SmarTest s’inscrit dans un périmètre fonctionnel bien défini, centré sur la création et la gestion des évaluations côté professeur, et le passage de ces évaluations côté étudiant. Le système prend en charge l’import de documents pédagogiques, la génération automatique de questions, la création et publication d’évaluations, la supervision en temps réel, le passage des évaluations, la correction automatique et la consultation des résultats.

1.4.2 Ce que le projet ne couvre pas

Certaines fonctionnalités ont été volontairement écartées du périmètre de ce projet :

- **La gestion administrative des utilisateurs** en masse ;
- **La surveillance anti-triche avancée** (détection de changement d’onglet, capture webcam, verrouillage du navigateur) ;
- **La gestion multi-professeurs** simultanément ;
- **L’application mobile** — seule une interface web responsive est prévue pour les étudiants ;
- **Les tableaux de bord analytiques poussés** (courbes de progression, comparaisons entre promotions).

1.5 Structure du rapport

Ce rapport est organisé en six chapitres. Le **Chapitre 1** pose le cadre du projet avec le contexte, la problématique et les objectifs. Le **Chapitre 2** dresse un état de l’art des solutions existantes et établit l’analyse des besoins. Le **Chapitre 3** décrit les choix architecturaux et les modélisations UML. Le **Chapitre 4** présente les fonctionnalités réalisées pour chaque composante du système. Le **Chapitre 5** expose la stratégie de tests et les résultats obtenus. Le **Chapitre 6** dresse le bilan et les perspectives d’évolution.

Chapitre 2 | Étude de l'Existant et Analyse des Besoins

2.1 Étude de l'existant

2.1.1 Solutions existantes

La gestion des quiz et examens en ligne n'est pas un domaine vierge : plusieurs plateformes sont aujourd'hui largement utilisées dans le milieu éducatif. Une analyse de ces solutions permet de mieux cerner leurs apports mais aussi leurs limites.

Moodle

Moodle est une plateforme d'apprentissage en ligne open source, très répandue dans les établissements d'enseignement supérieur. Elle propose un module d'évaluation complet permettant de créer des quiz avec différents types de questions. Cependant, elle ne propose pas de génération automatique de questions à partir de contenus pédagogiques, nécessite une infrastructure serveur lourde à maintenir, et ne garantit pas la confidentialité des données dans le cas d'un hébergement externe.

Google Forms

Google Forms est un outil de création de formulaires souvent utilisé à des fins d'évaluation. Simple et gratuit, il permet de créer rapidement des quiz avec correction automatique. Ses limites sont nombreuses : absence de génération intelligente de questions, contrôle très limité sur le déroulement de l'épreuve, et stockage intégral des données sur les serveurs de Google.

Kahoot !

Kahoot ! est une plateforme de quiz interactifs principalement conçue pour des sessions ludiques en classe. Elle offre une expérience engageante grâce à son interface gamifiée, mais elle est essentiellement orientée vers des évaluations informelles et ne convient pas aux examens formels.

Quizlet

Quizlet est un outil d'apprentissage basé sur des fiches et des mini-quiz, orienté vers l'autoapprentissage individuel et non vers la gestion d'examens collectifs. Les contenus importés transitent vers des serveurs externes, posant des problèmes de confidentialité.

Services LLM externes (ChatGPT, etc.)

Certains enseignants recourent à des services LLM externes pour générer des questions à partir de leurs cours. Si cette approche donne des résultats satisfaisants en termes de qualité, elle soulève un problème majeur : le contenu pédagogique est transmis vers des serveurs tiers non contrôlés.

2.1.2 Tableau comparatif critique

TABLE 2.1 – Comparaison des solutions existantes

Critère	Moodle	G. Forms	Kahoot !	Quizlet	SmarTest
Génération auto de questions	Non	Non	Non	Partiel	Oui
Contrôle temps réel	Partiel	Non	Oui	Non	Oui
Confidentialité des données	Partiel	Non	Non	Non	Oui
Stockage local des contenus	Non	Non	Non	Non	Oui
Interface desktop professeur	Non	Non	Non	Non	Oui
Correction automatique	Oui	Oui	Non	Oui	Oui
Statistiques détaillées	Oui	Non	Partiel	Non	Oui
Open source	Oui	Non	Non	Non	Oui

2.1.3 Limites identifiées

L'analyse comparative met en évidence plusieurs lacunes communes :

- Absence de génération intelligente intégrée à partir de documents pédagogiques ;
- Risques liés à la confidentialité des données hébergées chez des tiers ;
- Absence de contrôle en temps réel sur le déroulement des examens ;
- Inadaptation au contexte académique formel ;
- Manque de souveraineté sur les données pédagogiques.

C'est l'ensemble de ces lacunes que le projet SmarTest cherche à combler.

2.2 Analyse des besoins

2.2.1 Besoins fonctionnels

Besoins fonctionnels — Professeur

Le flux de travail du professeur suit les étapes suivantes :

- **Choix du type d'évaluation** : quiz (évaluation courte, accessible librement) ou examen (évaluation formelle, supervisée en temps réel).
- **Import du document pédagogique** : fichier PDF, DOCX ou TXT utilisé de manière temporaire pour la génération.
- **Import de la liste des étudiants autorisés** via un fichier CSV.
- **Configuration des paramètres** : nombre de questions, niveau de difficulté, type de questions, durée.
- **Génération automatique des questions** via l'IA à partir du document importé.
- **Édition manuelle des questions** après génération.
- **Validation et publication** de l'évaluation.

- **Lancement et supervision en temps réel** dans le cas d'un examen.
- **Consultation des résultats et statistiques** à l'issue de l'évaluation.
- **Export des résultats** pour archivage ou transmission administrative.

Besoins fonctionnels — Étudiant

- **Authentification** sécurisée.
- **Tableau de bord** : accès aux quiz et examens disponibles.
- **Passage d'un quiz** : tentative libre avec chronomètre.
- **Passage d'un examen** : accès uniquement après lancement par le professeur.
- **Soumission des réponses** : envoi sécurisé au backend.
- **Consultation des résultats** : score disponible dès validation par le professeur.

2.2.2 Besoins non fonctionnels

TABLE 2.2 – Besoins non fonctionnels

Critère	Description
Sécurité	Les questions, quiz et examens sont stockés localement et ne transitent jamais vers le backend. Le contenu du cours est transmis à l'IA uniquement le temps de la génération. L'accès à la plateforme est protégé par authentification.
Performance	Les temps de réponse doivent rester faibles à toutes les étapes. La communication en temps réel garantit une latence minimale lors des sessions d'examen.
Ergonomie	Le processus de création d'une évaluation doit être guidé et intuitif, sans formation technique préalable. L'interface web doit être simple et épurée pour l'étudiant.
Fiabilité	Le système doit garantir une haute disponibilité durant les sessions d'examen, avec gestion des erreurs et capacité de reprise sur incident sans perte de données.
Maintenabilité	Le code doit être structuré, documenté et modulaire afin de faciliter les évolutions futures.
Compatibilité	L'application web doit fonctionner sur les navigateurs modernes sans installation préalable côté étudiant.

2.3 Identification des acteurs

Un acteur représente tout élément extérieur au système qui interagit avec lui. L'analyse du processus d'utilisation de SmarTest permet d'identifier deux acteurs principaux : le **professeur** et l'**étudiant**.

2.3.1 Professeur

Le professeur est l'acteur central de l'application desktop. Il intervient selon trois rôles :

TABLE 2.3 – Résumé des rôles du professeur

Rôle	Actions principales
Créateur d'évaluations	Choisir le type d'évaluation, importer le cours et la liste des étudiants, configurer les paramètres, générer les questions, éditer, valider et publier
Superviseur d'examen	Lancer la session, suivre le déroulement en temps réel, clôturer l'épreuve
Analyste pédagogique	Consulter les résultats, analyser les statistiques, exporter les données

2.3.2 Étudiant

L'étudiant est l'acteur de l'application web. Son interaction est volontairement simple et guidée.

TABLE 2.4 – Résumé des rôles de l'étudiant

Rôle	Actions principales
Apprenant	S'authentifier, accéder au tableau de bord, passer un quiz ou un examen, soumettre ses réponses
Consultant de résultats	Consulter son score et le détail de ses résultats

2.3.3 Synthèse des interactions

TABLE 2.5 – Interactions acteurs – composantes du système

Acteur	Composante	Interaction
Professeur	Application desktop	Création, configuration et publication des évaluations
Professeur	Service IA	Génération automatique des questions
Professeur	Backend (temps réel)	Lancement et supervision des sessions d'examen
Professeur	Backend (REST)	Consultation et export des résultats
Étudiant	Application web	Passage des évaluations et consultation des résultats
Étudiant	Backend	Authentification, soumission des réponses, résultats

Chapitre 3 | Conception du Système

3.1 Architecture globale du système

L'architecture du système SmarTest a été conçue afin de répondre aux besoins de génération intelligente des quiz et examens, de communication temps réel, de supervision des étudiants et de synchronisation entre les différentes plateformes.

Le système adopte une **architecture hybride client-serveur** composée de cinq modules interconnectés :

- une application desktop destinée aux professeurs ;
- une application web destinée aux étudiants ;
- un backend centralisé ;
- une base de données relationnelle ;
- un service d'intelligence artificielle en ligne.

Cette architecture garantit une séparation claire des responsabilités, une bonne maintenabilité, une évolutivité satisfaisante et une communication fluide entre les composantes.

3.1.1 Description des composants

Client Desktop — Professeur

L'application desktop offre au professeur un environnement autonome pour gérer les cours, générer les quiz et examens, superviser les étudiants et piloter les sessions. Elle communique avec le backend via des requêtes classiques pour les opérations de gestion et via une connexion en temps réel pour les interactions durant les examens.

Client Web — Étudiant

L'application web permet aux étudiants de rejoindre les examens, d'attendre le lancement officiel, de répondre aux questions et de recevoir les mises à jour en temps réel.

Backend Centralisé

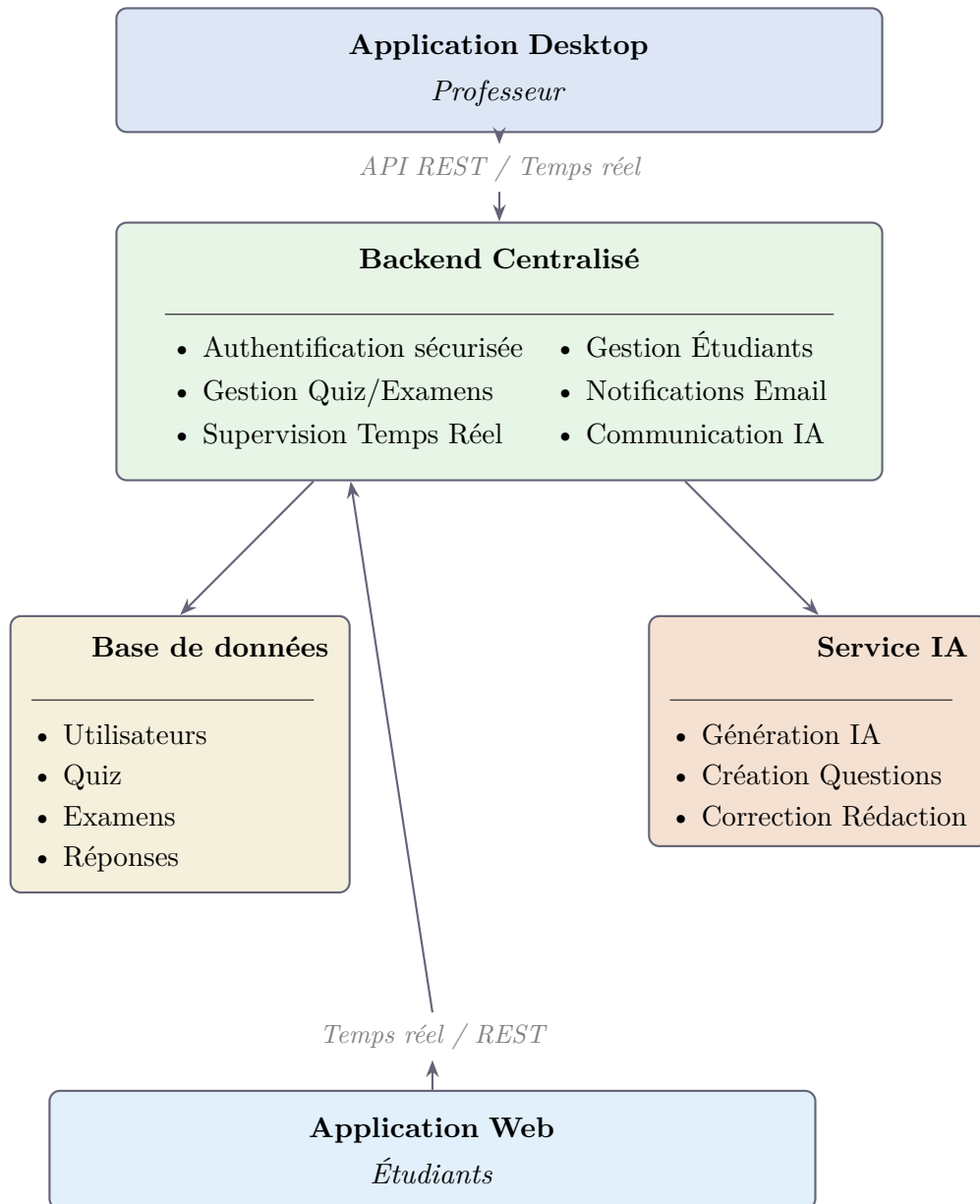
Le backend constitue le cœur du système. Il assure la gestion des données, l'authentification, la communication temps réel et la coordination entre les différentes composantes.

Service IA en ligne

Le projet utilise un service d'inférence rapide basé sur des grands modèles de langage pour générer les questions automatiquement et analyser les réponses de rédaction. Cette approche a été retenue après expérimentation de solutions locales qui présentaient des limites de performance.

3.1.2 Schéma d'architecture technique

FIGURE 3.1 – Architecture technique globale de SmarTest



3.2 Modélisation UML

Afin de mieux représenter le fonctionnement du système, plusieurs diagrammes UML ont été réalisés.

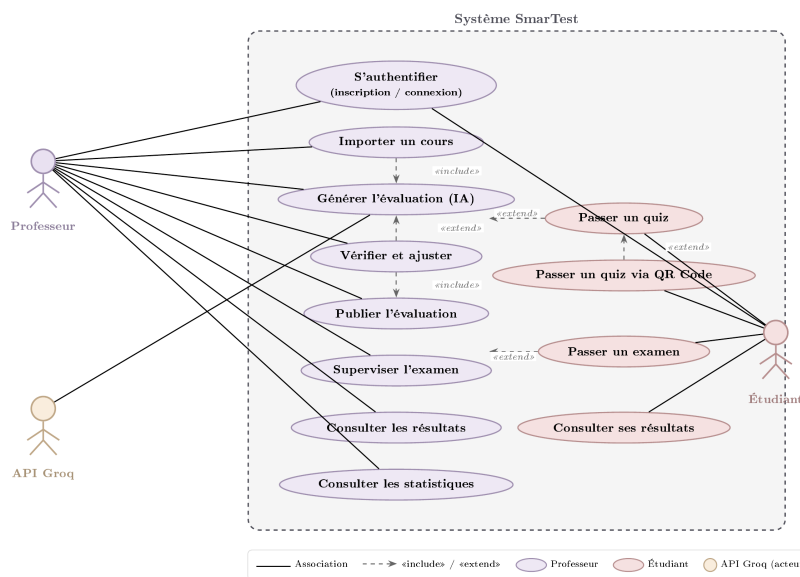
3.2.1 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation représente l'ensemble des interactions entre les acteurs et le système SmarTest. Il met en évidence la séparation des responsabilités entre le professeur, l'étudiant et le service d'intelligence artificielle.

TABLE 3.1 – Cas d'utilisation par acteur

Acteur	Cas d'utilisation
Professeur (Desktop)	S'inscrire, se connecter, gérer son compte, importer des cours, générer des questions via IA, créer et composer des quiz et examens, publier les évaluations, lancer et superviser les sessions en temps réel, corriger les rédactions avec l'IA, valider les notes et consulter les statistiques.
Étudiant (Web)	S'inscrire, se connecter, consulter son tableau de bord, passer un quiz avec correction immédiate, rejoindre une session d'examen, recevoir et répondre aux questions, consulter ses résultats.
Service IA	Générer automatiquement des questions à partir du contenu pédagogique, corriger les réponses de rédaction et proposer une note avec commentaire.

FIGURE 3.2 – Diagramme des cas d'utilisation — SmarTest



3.2.2 Diagramme de séquence

Les diagrammes de séquence décrivent l'ordre chronologique des échanges entre les composants du système pour chaque flux métier.

La figure 3.3 présente le diagramme de séquence global du système SmarTest, illustrant les sept flux principaux qui orchestrent l'ensemble des interactions entre les quatre composants : l'application desktop du professeur, l'API Groq Cloud, le backend Spring Boot et l'application web de l'étudiant.

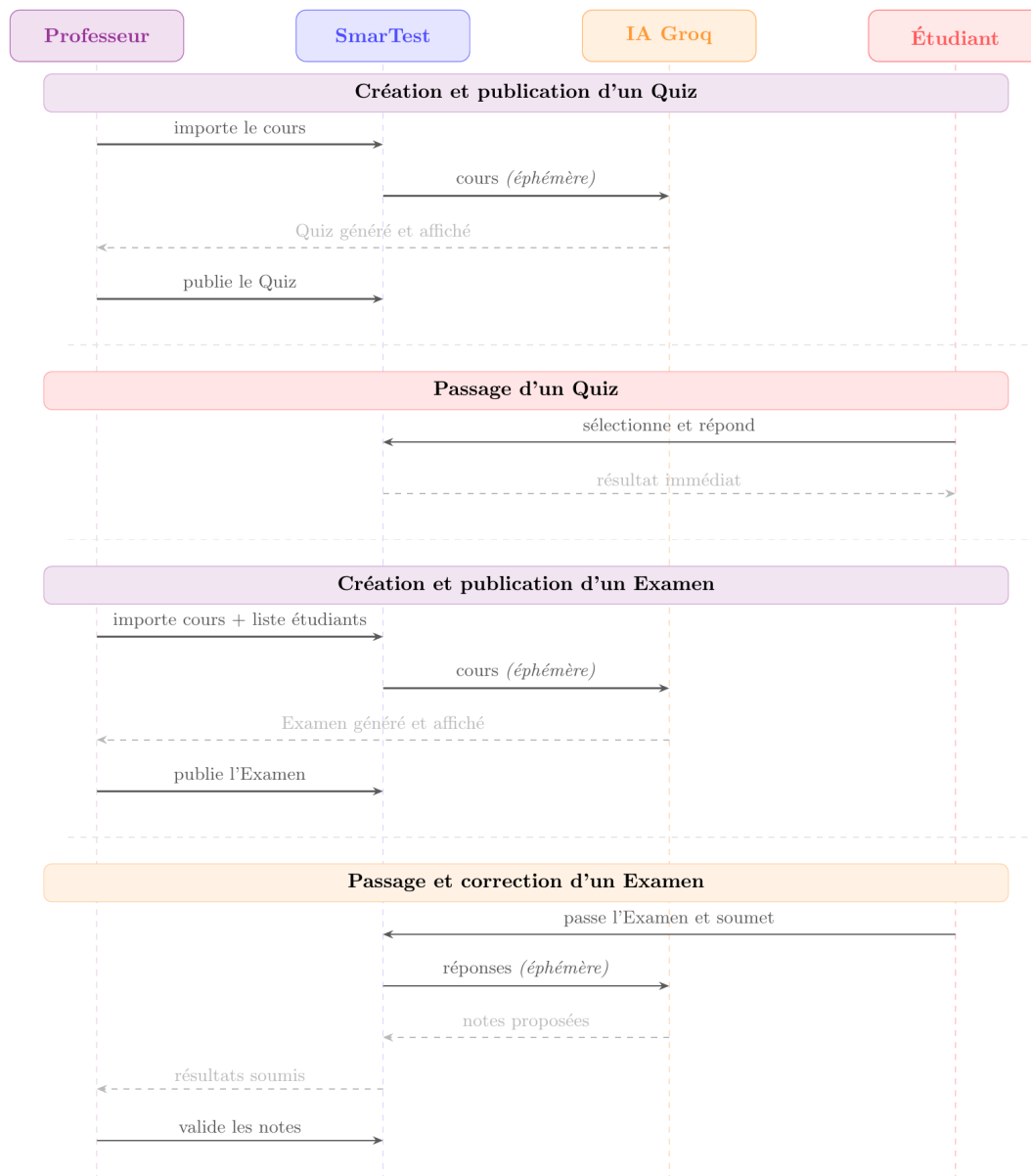


FIGURE 3.3 – Diagramme de séquence global du système SmarTest

3.2.3 Diagramme de classes

Le diagramme de classes global représente l'ensemble des entités métier du système SmarTest, leurs attributs essentiels, leurs méthodes principales et les relations qui les unissent. Il est organisé en deux couches distinctes conformément à l'architecture hybride du projet.

La **couche locale** (fond vert) regroupe les entités persistées en SQLite sur le poste du professeur : `CoursLocal`, `QuestionLocale`, `QuizLocal`, `ExamenLocal`, `SessionLocale` et `GroqService`. Ces entités ne quittent jamais le poste sauf lors de l'envoi éphémère du contenu pédagogique à Groq pour la génération de questions.

La **couche backend** (fond bleu) regroupe les entités persistées en MySQL via Spring Boot : la hiérarchie `Utilisateur` / `Professeur` / `Etudiant`, les évaluations publiées (`ExamenPublie`, `Quiz`, `Question`), et les données de session (`SessionExamen`, `Reponse`,

Resultat). Les questions ne sont jamais écrites en base MySQL : elles sont chargées en mémoire vive pendant la session et purgées à sa clôture.

Les flèches en pointillés gris illustrent le flux de *publication* : un QuizLocal ou un ExamenLocal est publié vers le backend (métadonnées et questions uniquement, sans le cours source).

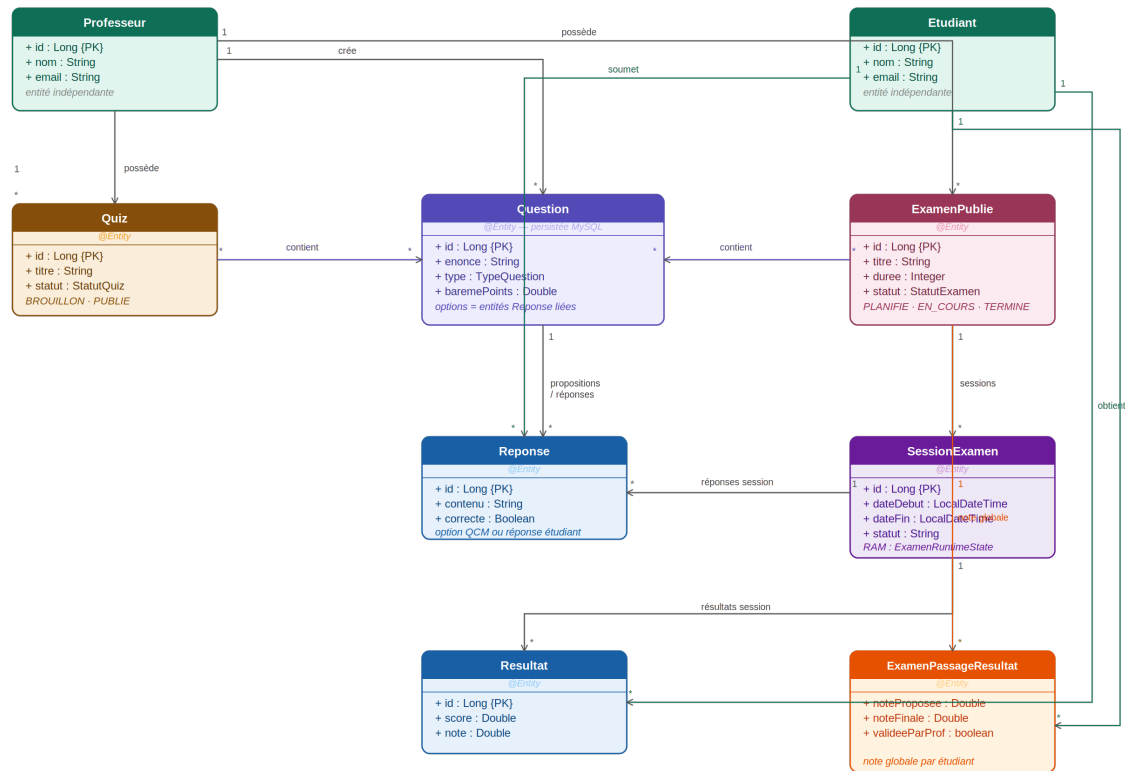


FIGURE 3.4 – Diagramme de classes global du système SmarTest

3.3 Conception des interfaces

3.3.1 Côté professeur — application desktop

TABLE 3.2 – Modules de l'application desktop (Professeur)

Module	Fonctionnalités
Tableau de bord	Vue synthétique des statistiques, accès rapide aux derniers examens, alertes sur taux d'échec.
Gestion des cours	Import de fichiers, extraction du contenu, stockage local.
Génération IA	Sélection du cours, choix du type, du nombre et du niveau des questions, génération, révision et validation manuelle.
Supervision	Liste des étudiants connectés en temps réel, suivi question par question, démarrage/arrêt des sessions, export des résultats.

3.3.2 Côté étudiant — application web

TABLE 3.3 – Écrans de l'application web (Étudiant)

Écran	Fonctionnalités
Authentification	Connexion sécurisée par email/mot de passe.
Tableau de bord	Liste des examens disponibles ou planifiés, statut de chaque évaluation.
Passage d'examen	Salle d'attente avant lancement, affichage dynamique question par question en temps réel, chronomètre, soumission sécurisée des réponses.
Résultats	Consultation des notes validées, détail par question, score global et commentaires de correction.

Chapitre 4 | Réalisation et Implémentation

La mise en œuvre de SmarTest repose sur une architecture distribuée présentée dans ce chapitre. Nous décrivons successivement les fonctionnalités réalisées pour les trois modules constitutifs du système : le client lourd du professeur, l'application web des étudiants et le backend. L'ensemble du développement est guidé par un principe de sécurité fondamental : les données pédagogiques sensibles restent sous le contrôle exclusif du professeur, stockées localement sur sa machine et jamais transmises à la base de données centrale.

4.1 Technologies utilisées

TABLE 4.1 – Stack technologique globale du projet

Composante	Technologie principale	Rôle
Application Desktop	C# / .NET 8 (WPF)	Interface professeur, stockage local, génération IA
Backend	Java / Spring Boot	API REST, authentification, supervision temps réel
Base de données	MySQL	Stockage des métadonnées fonctionnelles
Application Web	React / TypeScript	Interface étudiant, passage des évaluations
Service IA	API Groq (LLM)	Génération des questions, correction des rédactions
Stockage local	SQLite	Données pédagogiques exclusivement côté professeur

4.2 Justification des choix technologiques

Les technologies retenues pour SmarTest ont été choisies selon trois critères principaux : la robustesse technique, l'adéquation aux contraintes du projet (sécurité, temps réel, stockage local) et la maîtrise par l'équipe de développement.

TABLE 4.2 – Choix technologiques et justifications

Technologie	Justification
.NET 8 / WPF	Interface native Windows, accès direct au système de fichiers local, intégration aisée avec les services de chiffrement et les bibliothèques d'import de documents (PDF, DOCX, XLSX).
Spring Boot (Java 23)	Robustesse éprouvée, sécurité intégrée via Spring Security et JWT, support natif WebSocket/STOMP, et écosystème mature facilitant les tests d'intégration avec H2.
React + TypeScript	Interface réactive grâce au Virtual DOM, typage fort réduisant les erreurs, excellent support des connexions temps réel via SockJS/STOMP, et gestion d'état efficace avec Zustand.
MySQL 8	Fiabilité éprouvée, conformité ACID, bonne intégration avec Spring Data JPA, et adaptation à la charge prévue.
SQLite + AES-256	Indépendance totale vis-à-vis du cloud, contrôle intégral par l'enseignant sur ses données sensibles (cours, questions, examens), avec chiffrement via SQLCipher.
API Groq	Accès rapide et gratuit à des modèles LLM performants (LLaMA 3, Mixtral). Contrairement aux LLM locaux qui nécessitent un GPU dédié, Groq offre une puissance de calcul cloud avec une garantie contractuelle stipulant que les données transmises ne sont pas utilisées pour l'entraînement des modèles.
WebSocket / STOMP	Latence minimale, communication bidirectionnelle entre le backend et les clients, support natif dans Spring Boot et React via SockJS.
JWT	Mécanisme stateless compatible REST, sécurisé et standard, facilitant la gestion des rôles (professeur / étudiant) sans session serveur.
JUnit 5 + Mockito + H2	Isolation complète de la base de production, rollback automatique via <code>@Transactional</code> , et couverture des services métier critiques sans infrastructure externe.
Vitest + MSW	Simulation des appels HTTP sans serveur réel, rendu complet des pages React dans un environnement jsdom, et exécution rapide (12s pour 54 tests).
xUnit + Moq + EF Core InMemory	Isolation des dépendances externes, validation de la couche service/persistance sans base SQLite sur disque.

4.3 Fonctionnalités réalisées — Application Desktop

L'application desktop constitue le cœur pédagogique de SmarTest. Elle est la seule composante qui accède aux contenus de cours, génère les questions via l'IA et supervise les sessions d'examen.

4.3.1 Import de documents pédagogiques

Le professeur importe un fichier PDF, Word ou texte depuis son poste. Le contenu textuel est extrait et stocké exclusivement en local. Conformément au principe de confidentialité, le texte extrait reste sur le poste du professeur et n'est jamais transmis vers un

service externe.

4.3.2 Génération automatique de questions via l'IA

À partir d'un cours local sélectionné, le professeur configure les paramètres de génération (type de questions, difficulté, nombre). Le service IA génère les questions et les retourne structurées pour révision. Les questions sont affichées pour validation manuelle, puis sauvegardées en local. Le contenu du cours est transmis de manière éphémère pour la génération et n'est jamais persisté par le service IA.

4.3.3 Import de la liste des étudiants autorisés

Le professeur importe la liste des étudiants via un fichier CSV ou Excel. Le service valide chaque adresse e-mail et élimine les doublons. Pour l'examen, cet import est obligatoire ; pour le quiz, il est optionnel.

4.3.4 Publication vers le backend

Lors de la publication d'un quiz ou d'un examen, seules les métadonnées fonctionnelles transitent vers le backend : titre, durée, et liste des e-mails autorisés. Les contenus pédagogiques restent exclusivement dans le stockage local.

TABLE 4.3 – Séparation des données lors de la publication

Donnée	Stockage	Transmise au backend ?
Contenu du cours (texte)	Local	Non
Questions (énoncés, options)	Local	Non
Réponses modèles (rédaction)	Local	Non
Titre du quiz / examen	Backend	Oui
Durée (minutes)	Backend	Oui
Liste e-mails autorisés	Backend	Oui
Contenu cours (génération IA)	Jamais persisté	Éphémère

4.3.5 Supervision des sessions d'examen

Lors d'une session d'examen, le professeur contrôle la progression depuis l'interface de supervision. Les questions sont chargées depuis le stockage local par le desktop puis diffusées aux étudiants en temps réel. Le professeur visualise le nombre d'étudiants connectés et la progression des réponses pour chaque question.

4.3.6 Correction des examens

Après la clôture de la session, le professeur accède à la liste des réponses et peut lancer la correction. Pour les questions à choix (QCM, Vrai/Faux), la correction est automatique. Pour les questions de rédaction, le service IA compare la réponse de l'étudiant à la réponse modèle locale et retourne une note avec commentaire. Le professeur garde la main en validant ou modifiant la note avant enregistrement définitif.

4.4 Fonctionnalités réalisées — Application Web

L'application web est accessible aux étudiants depuis n'importe quel navigateur moderne.

4.4.1 Authentification et tableau de bord

L'étudiant se connecte de manière sécurisée. Son tableau de bord affiche uniquement les quiz et examens pour lesquels son e-mail figure dans la liste blanche définie par le professeur.

4.4.2 Passage des évaluations

Pour le quiz, l'étudiant répond aux questions et reçoit une correction immédiate à la fin. Pour l'examen, l'étudiant rejoint une salle d'attente, puis reçoit les questions en temps réel au fil de l'avancement contrôlé par le professeur. Un chronomètre est affiché pour chaque question. À l'expiration du temps, la réponse en cours est soumise automatiquement.

4.4.3 Consultation des résultats

Dès la validation par le professeur, l'étudiant accède à son score global et au détail de ses résultats par question, avec les commentaires de correction.

4.5 Fonctionnalités réalisées — Backend

Le backend assure l'authentification, le filtrage des accès par rôle, la communication temps réel et la persistance des données fonctionnelles.

4.5.1 Authentification et contrôle d'accès

Chaque requête est validée par vérification du jeton d'authentification. Les règles de contrôle d'accès garantissent que chaque acteur n'accède qu'aux ressources qui lui sont autorisées selon son rôle (professeur ou étudiant).

4.5.2 Supervision des examens

L'état de chaque session d'examen est géré en mémoire vive. Les questions ne sont jamais persistées en base de données : elles sont transmises lors du lancement et stockées exclusivement en mémoire jusqu'à la clôture de la session.

4.5.3 Correction automatique

Le backend assure la correction automatique des questions à choix et délègue au service IA la correction sémantique des réponses de rédaction.

TABLE 4.4 – Types de questions et méthode de correction

Type	Méthode de correction	Automatique ?
QCM	Correspondance avec la bonne réponse	Oui
Vrai / Faux	Correspondance avec la bonne réponse	Oui
Cases à cocher	Score proportionnel aux bonnes réponses	Oui
Rédaction	Évaluation sémantique par le service IA	Oui (avec validation professeur)

4.6 Gestion de la sécurité

La sécurité de SmarTest est conçue selon le principe du moindre privilège et de la séparation stricte des données.

- **Stockage local des données pédagogiques** : les questions, quiz et examens ne quittent jamais le poste du professeur.
- **Mémoire vive pour les sessions** : les questions durant un examen sont stockées uniquement en mémoire et purgées à la clôture de la session.
- **Liste blanche des e-mails** : seuls les étudiants autorisés peuvent rejoindre une évaluation. Tout accès non autorisé est refusé.
- **Minuteur bloquant côté navigateur** : le compte à rebours soumet automatiquement une réponse vide à l'expiration du temps, empêchant toute extension de délai.
- **Progression contrôlée par le professeur** : c'est l'application desktop du professeur qui pilote l'avancement des questions. Les étudiants ne peuvent pas naviguer librement.
- **Soumission unique** : le système refuse les réponses multiples pour un même couple étudiant/question dans une session.

4.6.1 Synthèse de l'implémentation

TABLE 4.5 – Récapitulatif de l'implémentation par composante

Composante	Fonctionnalités réalisées
Desktop	Import de documents, génération de questions par IA, stockage local sécurisé, publication, supervision des sessions, correction assistée par IA, gestion du compte
Backend	Authentification, notifications email, gestion des quiz et examens, supervision temps réel, correction IA des rédactions, sessions quiz rapide
Frontend	Authentification, tableau de bord, passage de quiz et examens, supervision, consultation des résultats et corrections

4.7 Statistiques et alertes pédagogiques

Le module de statistiques de SmarTest couvre trois flux distincts — quiz persisté, examen publié et session QR live — avec des règles de calcul propres à chaque contexte. Le professeur consulte l'ensemble de ces données depuis l'application desktop, qui les affiche sous forme de tableau de bord enrichi.

4.7.1 Statistiques de quiz (flux persisté)

Les statistiques officielles d'un quiz reposent exclusivement sur la **première tentative** de chaque étudiant, identifiée avant soumission : si l'étudiant possède déjà un résultat pour ce quiz (`existsByEtudiantIdAndQuizId`), les nouvelles lignes sont enregistrées avec le drapeau `estPremiereTentative = false` et exclues du calcul.

Indicateurs par question. Pour chaque question du quiz, le service calcule :

- **nombreReponses** : total des réponses en première tentative ;
- **pourcentageReussite** et **pourcentageEchec** ;
- une **alerte automatique** persistée dans `statistique_question` lorsque le taux d'échec atteint ou dépasse **70 %** (seuil `SEUIL_ALERTE_ECHEC_POURCENT`).

Agrégats quiz. Les agrégats suivants sont calculés à l'échelle du quiz :

- **nombreParticipants** : étudiants distincts ayant au moins une réponse en première tentative ;
- **tauxReussiteGlobal** : moyenne arithmétique des taux de réussite par question (non pondérée par le nombre de réponses) ;
- **moyenneNoteSur20** : pour chaque étudiant, $\text{note} = (\text{bonnes} / \text{réponses enregistrées}) \times 20$, puis moyenne sur l'ensemble des participants ;
- **repartitionParTranche** : histogramme sur sept tranches (0–5, 5–8, 8–10, 10–12, 12–15, 15–18, 18–20).

Déclenchement et persistance. Le recalcul est déclenché de deux façons complémentaires :

1. **Automatiquement**, trois secondes après chaque soumission, via un appel asynchrone à `StatistiqueRecalculService` ;
2. **À la demande**, lors de chaque appel REST du professeur (`GET /api/statistiques/quiz/{quizId}`) qui recalcule et persiste les données avant de les retourner.

En parallèle, un message WebSocket est publié immédiatement après chaque soumission sur le topic `/topic/quiz/{quizId}/stats`, permettant à l'interface desktop de se rafraîchir en temps réel.

4.7.2 Statistiques d'examen publié

Les statistiques d'examen partagent le même format de réponse (`StatistiquesQuizResponse`), mais leur source et leurs règles diffèrent :

- La source est `ExamenPassageResultat` et `ExamenCorrectionLigne`, et non la table `Resultat quiz` ;

- Le calcul est effectué à la volée (lecture seule) et **non persisté** dans `statistique_question` ;
- Une question est considérée réussie si le score effectif est supérieur ou égal à **50 %** des points alloués, ce qui prend en charge les notes partielles (questions à cases à cocher) ;
- Les notes sur 20 sont ramenées au barème de référence avant application des mêmes tranches d'histogramme.

4.7.3 Statistiques live QR (en mémoire)

Les sessions QR éphémères gèrent leurs statistiques exclusivement **en mémoire vive** ; aucune donnée n'est écrite en base MySQL. Deux services coexistent :

- **QuizSessionManager** : agrège les réponses par question et déduplique les participants par clé participant. Le taux global est la moyenne des taux de réussite par question, identique au calcul des stats quiz persistées. Les données sont effacées à la clôture de la session. Les mises à jour sont diffusées via STOMP sur `/topic/quiz-qr/{sessionToken}/stream` ;
- **QuizQrLiveStatsService** : agrège toutes les vérifications liées à un `quizId` (sans notion de première tentative). Le taux global est ici pondéré : `totalCorrectes / totalSoumissions`. Les données sont exposées sur `/topic/quiz/{quizId}/qr-live`.

4.7.4 Affichage desktop et seuils d'alerte

L'application desktop (`StatistiquesQuizProfViewModel`) interroge l'API toutes les **huit secondes** en arrière-plan (sans indicateur de chargement pour les rafraîchissements silencieux) et affiche :

- la moyenne sur 20, le taux global et l'histogramme de répartition ;
- des barres par question dont la couleur évolue du rouge au vert selon le pourcentage de réussite ;
- une bannière d'alerte et une icône par question dont le taux d'échec dépasse **40 %** (`SeuilPourcentageEchecAlerte`) ;
- un export des données au format CSV ou PDF.

TABLE 4.6 – Seuils d'alerte — comparaison backend / interface desktop

Niveau	Seuil d'échec	Effet
Backend (<code>StatistiqueService</code>)	70 %	Alerte persistée dans <code>statistique_question</code>
Desktop (<code>SeuilPourcentageEchecAlerte</code>)	40 %	Bannière et icône dans l'interface professeur

Cet écart volontaire permet au professeur d'être alerté visuellement en amont, avant que la question n'atteigne le seuil critique côté serveur.

4.7.5 Synthèse du flux statistique (quiz persisté)

La figure 4.1 illustre le parcours complet des données statistiques pour un quiz persisté, depuis la soumission de l'étudiant jusqu'à l'affichage sur le desktop du professeur.

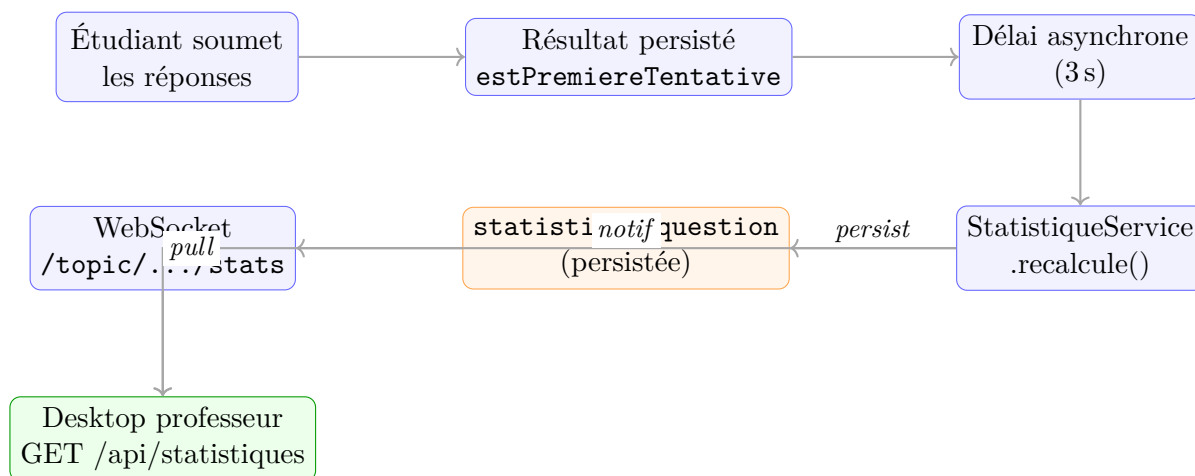


FIGURE 4.1 – Flux des statistiques pour un quiz persisté

Chapitre 5 | Tests et Validation

5.1 Stratégie de tests

La stratégie de tests adoptée pour SmarTest repose sur une approche en couches, couvrant les trois composants de l'application : le backend Spring Boot, le frontend React/Vite et l'application desktop WPF/.NET. L'objectif principal est de garantir la fiabilité fonctionnelle de chaque module tout en respectant une contrainte absolue : aucune interaction avec la base de données de production ne doit avoir lieu lors de l'exécution des tests. Pour cela, trois technologies d'isolation ont été utilisées : H2 in-memory pour le backend, EF Core InMemory pour le desktop, et MSW (Mock Service Worker) pour le frontend. Au total, **466 tests automatisés** ont été écrits et exécutés : 309 tests backend, 54 tests frontend et 103 tests desktop.

5.2 Tests unitaires

Les tests unitaires valident le comportement isolé de chaque composant métier, toutes les dépendances étant substituées par des mocks.

Backend – JUnit 5 et Mockito : les composants couverts incluent AuthService (inscription, connexion, génération et validation JWT), QuizService (création, correction automatique, calcul de statistiques), SessionExamenService, JwtAuthFilter (token valide, expiré, absent, malformé) et GlobalExceptionHandler.

Frontend – Vitest et Testing Library : les tests couvrent les hooks Zustand (useAuth : connexion, déconnexion, persistance localStorage), les intercepteurs Axios (gestion des erreurs 401, 403, 429) et les composants React isolés (QuizCard, PrivateRoute).

Desktop – xUnit et Moq : les tests couvrent CryptoService (chiffrement/déchiffrement round-trip), ApiFailureParser (mapping des codes HTTP vers les messages utilisateur), AuthService et les ViewModels (ExamGenerationViewModel, ExamCorrectionViewModel).

5.3 Tests d'intégration

Les tests d'intégration vérifient le comportement de plusieurs composants assemblés, en s'appuyant sur des bases de données en mémoire et des serveurs HTTP simulés.

Backend : une classe de base `BaseIntegrationTest` annotée `@SpringBootTest`, `@AutoConfigureMockMvc` et `@Transactional` est utilisée. Le profil `test` active H2 in-memory et désactive Flyway. Les contrôleurs testés incluent `AuthController`, `QuizController`, `SessionExamenController` et `ExamenPublieController`.

Frontend : les pages React complètes sont rendues dans un environnement jsdom avec MSW interceptant tous les appels HTTP. Les scénarios couverts incluent Login (succès, 401, 403), Register (validation domaine, conflit 409) et QuizPassageWeb (chargement, soumission, score final, restauration depuis sessionStorage).

Desktop : les tests utilisent `TestDbFactory.CreateInMemory()` pour obtenir un `LocalDbContext` isolé par test, couvrant `LocalQuizService` (CRUD, recherche, pagination) et `ImportService` (XLSX, TXT, PDF).

5.4 Tests fonctionnels

Scénario principal — Un professeur crée un examen → un étudiant le passe :

1. Le professeur s'authentifie via POST /auth/login, le token JWT est stocké.
2. Il crée un examen avec plusieurs QCM via POST /api/examens.
3. Il publie l'examen via PUT /api/examens/{id}/publier, le rendant accessible aux étudiants.
4. L'étudiant s'authentifie et accède à l'examen (cas 403 et 404 également couverts).
5. L'étudiant sélectionne ses réponses, soumet et consulte son score immédiatement.

Ce scénario confirme que les trois couches de l'application (authentification, logique métier, interface utilisateur) fonctionnent correctement de manière coordonnée.

5.5 Tests de performance et de charge

Backend : un test JUnit simule 50 utilisateurs envoyant simultanément une requête de connexion via MockMvc. Le seuil d'acceptation est fixé à 90% de succès minimum (45/50). Un script Gatling complémentaire cible p95 < 500 ms et un taux d'erreur < 1%.

Frontend : un test CPU mesure le tri de 20 000 entiers, avec un seuil fixé à 200 ms.

Desktop : les opérations critiques sont mesurées sur EF Core InMemory — insertion de 10 000 quiz (< 3 s), recherche textuelle sur 10 000 enregistrements (< 100 ms), import XLSX de 500 lignes (< 2 s).

5.6 Résultats et corrections apportées

L'ensemble des 466 tests a été exécuté avec succès, sans aucun échec :

TABLE 5.1 – Récapitulatif des résultats de tests

Package	Total	Réussis	Échecs	Durée
Backend Spring Boot	309	309	0	28 s
Frontend React/Vite	54	54	0	12 s
Desktop WPF/.NET	103	103	0	4,6 s
Total	466	466	0	≈45 s

Plusieurs corrections ont été apportées durant le développement : désactivation de Flyway dans le profil test pour résoudre l'incompatibilité de syntaxe MySQL/H2, décomantage et réécriture de tests backend inactifs (SessionExamenServiceTest, ExamenControllerTest), et séparation du projet de tests desktop du projet WPF principal pour éviter les conflits de génération XAML.

Chapitre 6 | Conclusion et Perspectives

6.1 Bilan du projet : objectifs atteints

Au terme de ce projet, l'équipe de développement dresse un bilan globalement positif. L'ensemble des objectifs fixés en début de projet a été atteint, et le système SmarTest est aujourd'hui une plateforme fonctionnelle, sécurisée par authentification JWT et prête à l'emploi dans un contexte académique réel.

6.1.1 Objectifs fonctionnels atteints

Le processus complet de création et de gestion des évaluations a été implémenté avec succès, de l'import du document pédagogique jusqu'à la consultation des résultats par l'étudiant. Les fonctionnalités suivantes ont été intégralement réalisées :

- **Processus de création guidé** : le professeur peut, depuis l'application desktop, choisir le type d'évaluation, importer un document pédagogique et la liste des étudiants concernés, configurer les paramètres de génération (nombre de questions, type, difficulté, durée), générer automatiquement les questions, les éditer manuellement, puis valider et publier l'évaluation .
- **Génération automatique de questions** : l'intégration de l'API Groq permet de générer des questions pertinentes et variées à partir du contenu du cours importé, avec un temps de réponse rapide et des résultats de qualité satisfaisante .
- **Gestion différenciée quiz et examen** : le système distingue clairement les deux types d'évaluation. Le quiz est accessible librement par l'étudiant sur le web (plusieurs tentatives possibles) ; l'examen est lancé et supervisé en temps réel par le professeur via WebSocket .
- **Interface web étudiant** : l'application web développée avec React et TypeScript offre une expérience fluide et intuitive. L'étudiant peut s'authentifier, accéder à son tableau de bord, passer ses évaluations et consulter ses résultats .
- **Supervision en temps réel** : le professeur dispose d'un contrôle direct sur le déroulement des sessions d'examen (lancement, pause, navigation entre les questions, suivi des réponses) grâce à la communication WebSocket .
- **Correction automatique et statistiques** : les réponses de type QCM et vrai/faux sont corrigées automatiquement par comparaison avec la bonne réponse enregistrée en base. En passage web, l'étudiant peut vérifier chaque question au fil du quiz, puis obtient un score global à la soumission finale. Pour les examens, les questions fermées sont traitées de façon déterministe, tandis que les rédactions font l'objet d'une correction différée par intelligence artificielle (API Groq) ; la note n'est communiquée à l'étudiant qu'après validation par le professeur. Le professeur dispose, dans l'application bureau, de statistiques agrégées et d'indicateurs détaillés par question (taux de réussite, nombre de réponses correctes et incorrectes, moyenne sur 20, répartition des notes par tranches). Pour les quiz, ces indicateurs s'appuient sur la première tentative de chaque participant. Le système signale automatiquement les questions dont le taux d'échec atteint ou dépasse 70 %, afin d'identifier les points de cours à retravailler .

- **Architecture des données** : les cours et les brouillons d'évaluations sont stockés localement sur le poste du professeur (base SQLite). Lors de la publication, les quiz et examens sont synchronisés vers le backend afin de permettre le passage et le suivi côté étudiants. Le contenu du cours est transmis à l'API Groq lors de la génération des questions ; les rédactions d'examen peuvent également être envoyées à Groq pour la correction côté serveur.

6.1.2 Objectifs non fonctionnels atteints

Au-delà des fonctionnalités, les exigences de qualité fixées en début de projet ont été satisfaites :

TABLE 6.1 – Bilan des objectifs non fonctionnels

Critère	Objectif fixé	Résultat obtenu
Sécurité	Accès authentifié, données protégées	Authentification JWT, rôles professeur/étudiant, contrôle d'accès aux ressources publiées
Performance	Temps de réponse rapides, latence minimale	Génération Groq fluide, WebSocket réactif durant les sessions
Ergonomie	Interfaces intuitives sans formation préalable	Processus guidé côté desktop, interface épurée côté web
Fiabilité	Haute disponibilité durant les examens	Sessions stables, repli HTTP si WebSocket indisponible
Compatibilité	Navigateurs modernes sans installation	Application web compatible Chrome, Firefox, Edge

6.1.3 Synthèse

Le tableau 6.2 dresse un récapitulatif de l'ensemble des fonctionnalités prévues et de leur état de réalisation à l'issue du projet.

TABLE 6.2 – Synthèse des fonctionnalités réalisées

Fonctionnalité	État
Import du document pédagogique	Réalisé
Import de la liste des étudiants	Réalisé
Configuration des paramètres	Réalisé
Génération automatique via Groq	Réalisé
Édition manuelle des questions	Réalisé
Validation et publication	Réalisé
Gestion des quiz (accès libre)	Réalisé
Lancement et supervision d'examen	Réalisé
Communication temps réel (WebSocket)	Réalisé
Correction automatique et calcul des scores	Réalisé
Statistiques et alertes	Réalisé
Export des résultats (CSV/PDF)	Réalisé
Tableau de bord étudiant	Réalisé
Authentification JWT	Réalisé
Application mobile	Perspective future
Anti-triche avancé	Perspective future

En définitive, SmarTest constitue une réponse concrète et opérationnelle aux problématiques identifiées en début de projet. Il démontre qu'il est possible de combiner intelligence artificielle, architecture client-serveur et interactivité en temps réel dans une plateforme d'évaluation académique.

6.2 Difficultés rencontrées et solutions adoptées

Le développement de SmarTest a été jalonné de plusieurs difficultés techniques et organisationnelles. Cette section présente les principaux obstacles rencontrés ainsi que les solutions adoptées pour les surmonter.

enumitem

6.2.1 Difficultés techniques

- **Intégration de l'IA locale** : l'intégration d'un agent IA local (Llama.cpp / GPT4All) au sein de l'application desktop .NET s'est avérée trop complexe et trop gourmande en ressources pour être réalisée dans le cadre de ce projet. Les modèles locaux testés produisaient des questions de qualité insuffisante et les temps de chargement étaient rédhibitoires. **Solution adoptée** : remplacement par l'API Groq,

qui offre des temps d'inférence très faibles et une qualité de génération nettement supérieure .

- **Communication en temps réel via WebSocket** : la mise en place de la communication bidirectionnelle entre le backend et les navigateurs des étudiants via WebSocket a nécessité une gestion fine des états de session, des connexions simultanées et des cas de déconnexion inattendue. **Solution adoptée** : reconnexion automatique côté client, suivi des états en mémoire côté serveur et repli par requêtes HTTP lorsque le canal temps réel est indisponible .
- **Synchronisation locale et serveur** : concilier le stockage local des brouillons (application desktop) avec la publication des évaluations sur le backend a nécessité une conception rigoureuse des flux de synchronisation (quiz, examens, listes d'e-mails autorisés). **Solution adoptée** : services dédiés à la publication web et contrôle de propriété des ressources côté serveur .
- **Extraction et traitement du contenu des cours** : l'extraction du texte à partir de fichiers PDF, DOCX et TXT de formats variés s'est avérée problématique, notamment pour les PDF contenant des images, des tableaux ou des mises en page complexes. **Solution adoptée** : extraction automatique selon le format du fichier, puis envoi du texte à l'API Groq pour la génération des questions .
- **Gestion du temps et synchronisation** : assurer la cohérence du chronomètre entre le serveur et les différents clients (web et bureau) tout en gérant les décalages réseau a représenté un défi non trivial. **Solution adoptée** : le temps de référence est géré côté serveur et diffusé aux clients via WebSocket .
- **Interopérabilité desktop et web** : faire communiquer une application desktop .NET (WPF) avec un backend Spring Boot et une interface web React a nécessité une conception soignée de l'API REST et des contrats d'interface clairs entre les composantes. **Solution adoptée** : définition d'une API REST commune et tests d'intégration sur les principaux parcours (authentification, publication, passage).

6.2.2 Difficultés organisationnelles

- **Coordination entre les membres de l'équipe** : le projet impliquait le développement simultané de trois composantes distinctes (desktop, web, backend), réparties entre les membres de l'équipe. La synchronisation des avancements et la gestion des dépendances entre les composantes ont parfois ralenti le développement. **Solution adoptée** : adoption d'un workflow Git structuré avec des branches dédiées par fonctionnalité et des réunions de synchronisation régulières .
- **Gestion du calendrier** : la découverte tardive des limitations de l'IA locale a contraint l'équipe à revoir une partie de l'architecture en cours de développement, ce qui a impacté le planning initial. **Solution adoptée** : réorganisation des priorités et basculement rapide vers l'API Groq, permettant de rattraper le retard accumulé sans compromettre la qualité du livrable final.

6.3 Perspectives d'évolution

SmarTest, dans sa version actuelle, constitue une base solide et fonctionnelle. Plusieurs axes d'évolution sont envisagés pour enrichir et améliorer la plateforme dans ses versions futures.

6.3.1 Intégration d'un agent IA véritablement local

L'objectif initial du projet était d'exécuter le modèle de génération de questions directement sur le poste du professeur, sans appel externe systématique. Cette perspective reste une priorité pour les versions futures. Avec l'évolution des modèles de langage légers (Mistral, Phi-3, Llama 3), il devient progressivement possible d'envisager un LLM local sur du matériel grand public. L'intégration de solutions telles qu'**Ollama** ou **LM Studio** au sein de l'application desktop constitue une piste concrète pour limiter l'envoi du contenu pédagogique vers le cloud.

6.3.2 Application mobile

Une application mobile destinée aux étudiants permettrait d'étendre l'accessibilité de SmarTest au-delà du navigateur web. Développée avec **React Native** ou **.NET MAUI** pour une compatibilité Android et iOS, elle offrirait les mêmes fonctionnalités que l'interface web (tableau de bord, passage de quiz et d'examens, consultation des résultats) avec une expérience utilisateur optimisée pour les écrans mobiles.

6.3.3 Mécanismes anti-triche avancés

La version actuelle intègre déjà des mécanismes de base : gestion du temps, liste d'étudiants autorisés, mélange des questions et des réponses, verrouillage des réponses après validation en examen, et pilotage de la session par le professeur. Des fonctionnalités plus avancées sont envisagées :

- détection de changement d'onglet ou de fenêtre durant l'examen
- verrouillage du navigateur en mode plein écran
- journalisation des comportements suspects
- analyse des temps de réponse pour détecter des anomalies

6.3.4 Tableau de bord analytique enrichi

Les statistiques actuelles fournissent des indicateurs par question, une moyenne sur 20 et une répartition par tranches, avec alerte lorsque le taux d'échec dépasse 70 %. Des analyses plus poussées pourraient être intégrées :

- suivi nominatif détaillé par étudiant
- courbes de progression sur plusieurs évaluations
- comparaison des performances entre groupes ou promotions
- rapports pédagogiques enrichis et exportables

6.3.5 Gestion multi-professeurs

La version actuelle associe chaque quiz et chaque examen à un seul professeur. Une évolution vers une gestion multi-enseignants permettrait à plusieurs professeurs de partager la même infrastructure backend, avec des espaces de travail isolés, une gestion des droits par rôle et un module d'administration centralisé.

6.3.6 Amélioration de la génération de questions

Plusieurs améliorations sont envisageables sur la qualité de la génération :

- enrichissement des types de questions et des formats d'évaluation
- génération ciblée par chapitre ou par notion clé extraite automatiquement du cours
- renforcement de l'évaluation automatique de la pertinence des questions générées avant validation par le professeur